Ain Shams University, Faculty of Engineering

Web Development
Calculator Lab Task

Group 1 Section 1
Mohamed Ashraf Hatem Eltaie 24p0024

calculator.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>iOS Calculator</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <div class="calculator">
        <div class="calculator-header">iOS Calculator</div>

        <!-- Calculator Screen -->
        <div id="display" class="calculator-screen">0</div>

        <!-- Calculator Keypad Grid -->
        <div class="calculator-keys">
            <button class="key action" data-action="clear" data-key="Escape">AC</button>
            <button class="key action" data-action="double-zero" data-key="00">00</button>
            <button class="key action" data-action="percent" data-key="%">%</button>
            <button class="key operator" data-action="divide" data-key="/">&divide;</button>

            <button class="key number" data-value="7" data-key="7">7</button>
            <button class="key number" data-value="8" data-key="8">8</button>
            <button class="key number" data-value="9" data-key="9">9</button>
            <button class="key operator" data-action="multiply" data-key="*">&times;</button>

            <button class="key number" data-value="4" data-key="4">4</button>
            <button class="key number" data-value="5" data-key="5">5</button>
            <button class="key number" data-value="6" data-key="6">6</button>
            <button class="key operator" data-action="subtract" data-key="-">-</button>

            <button class="key number" data-value="1" data-key="1">1</button>
            <button class="key number" data-value="2" data-key="2">2</button>
            <button class="key number" data-value="3" data-key="3">3</button>
            <button class="key operator" data-action="add" data-key="+">+</button>

            <button class="key number zero" data-value="0" data-key="0">0</button>
            <button class="key number" data-value="." data-key=".">.</button>
            <button class="key operator" data-action="calculate" data-key="Enter">=</button>
        </div>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

style.css

```
/* Reset and Base Styles */
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-
serif;
}

body {
    background-color: #f0f2f5;
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 100vh;
}

/* Calculator Container */
.calculator {
    background-color: #000000;
    width: 320px;
    border-radius: 24px;
    padding: 20px 16px 24px 16px;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.3);
}

/* Header Text */
.calculator-header {
    color: #ffffff;
    text-align: center;
    font-size: 1.1rem;
    font-weight: 500;
    margin-bottom: 16px;
}

/* Screen Display */
.calculator-screen {
    background-color: #000000;
    color: #ffffff;
    font-size: 2.8rem;
    font-weight: 300;
    text-align: right;
    padding: 15px 12px;
    margin-bottom: 20px;
    width: 100%;
    height: 75px;
    overflow-x: auto;
    overflow-y: hidden;
    white-space: nowrap;
    direction: ltr;
}
/* Style scrollbars for the display */
.calculator-screen::-webkit-scrollbar {
    height: 5px;
}
.calculator-screen::-webkit-scrollbar-thumb {
```

```
    background: #ff9f0a;
    border-radius: 3px;
}

/* Keypad Grid Layout */
.calculator-keys {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 12px;
}

/* Button Styles */
.key {
    background-color: #333333;
    color: #ffffff;
    font-size: 1.4rem;
    font-weight: 400;
    height: 60px;
    border: none;
    border-radius: 50%;
    cursor: pointer;
    display: flex;
    justify-content: center;
    align-items: center;
    transition: filter 0.2s;
    user-select: none;
}

.key:active, .key.keyboard-active {
    filter: brightness(130%);
}

/* Special Action Buttons (AC, 00, %) - Light Gray */
.key.action {
    background-color: #a5a5a5;
    color: #000000;
    font-weight: 500;
}

/* Operator Buttons (+, -, x, /, =) - Orange */
.key.operator {
    background-color: #ff9f0a;
    color: #ffffff;
    font-size: 1.6rem;
    font-weight: 500;
}

/* Number Buttons - Dark Gray */
.key.number {
    background-color: #333333;
}

/* Zero Button spanning two columns */
.key.zero {
    grid-column: span 2;
    border-radius: 30px;
    justify-content: flex-start;
    padding-left: 24px;
}
```

script.js

```
// DOM: Select nodes
const display = document.getElementById('display');
const keys = document.querySelector('.calculator-keys');
const allButtons = document.querySelectorAll('.key');

// Calculator State
let expression = '0';
let evaluated = false;

// Update screen display and automatically scroll to the end
function updateDisplay() {
    display.textContent = expression;
    display.scrollLeft = display.scrollWidth;
}

// Logic: Handle number and decimal inputs
function inputNumber(digit) {
    if (evaluated) {
        expression = digit;
        evaluated = false;
        return;
    }

    if (expression === '0' && digit !== '.') {
        expression = digit;
    } else {
        const tokens = expression.split(/[\+\-\×\÷]/);
        const currentToken = tokens[tokens.length - 1];
        if (digit === '.' && currentToken.includes('.')) return;

        expression += digit;
    }
}

// Logic: Handle operator inputs
function handleOperator(opSymbol) {
    if (evaluated) {
        evaluated = false;
    }

    const lastChar = expression.slice(-1);
    const operators = ['+', '-', '×', '÷'];

    if (operators.includes(lastChar)) {
        expression = expression.slice(0, -1) + opSymbol;
    } else {
        expression += opSymbol;
    }
}

// Logic: Reset calculator
function resetCalculator() {
    expression = '0';
    evaluated = false;
}
```

```
// Logic: Handle percentage conversion
function handlePercent() {
    try {
        const tokens = expression.split(/([\+\-\×\÷])/);
        let lastToken = tokens[tokens.length - 1];
        if (lastToken && !isNaN(lastToken)) {
            let val = parseFloat(lastToken) / 100;
            tokens[tokens.length - 1] = val.toString();
            expression = tokens.join('');
        }
    } catch (e) {
        expression = 'Error';
    }
}

// Logic: Evaluate expression with proper Operator Precedence
function evaluateExpression() {
    try {
        let cleanExpr = expression;
        const operators = ['+', '-', '×', '÷'];
        while (cleanExpr.length > 0 && operators.includes(cleanExpr.slice(-1))) {
            cleanExpr = cleanExpr.slice(0, -1);
        }

        let formattedExpr = cleanExpr.replace(/×/g, '*').replace(/÷/g, '/');

        let tokens = [];
        let currentNum = '';

        for (let i = 0; i < formattedExpr.length; i++) {
            let char = formattedExpr[i];
            if ((char === '-' || char === '+') && (i === 0 || ['*', '/', '+', '-
'].includes(formattedExpr[i - 1]))) {
                currentNum += char;
            } else if (['+', '-', '*', '/'].includes(char)) {
                if (currentNum !== '') {
                    tokens.push(parseFloat(currentNum));
                    currentNum = '';
                }
                tokens.push(char);
            } else {
                currentNum += char;
            }
        }
        if (currentNum !== '') {
            tokens.push(parseFloat(currentNum));
        }

        // Pass 1: Multiplication and Division
        let pass1 = [];
        for (let i = 0; i < tokens.length; i++) {
            if (tokens[i] === '*' || tokens[i] === '/') {
                let prev = pass1.pop();
                let next = tokens[i + 1];
                let res = tokens[i] === '*' ? prev * next : (next === 0 ? 'Error' : prev / next);
                if (res === 'Error') return 'Error';
                pass1.push(res);
                i++;
            } else {
```

```
                pass1.push(tokens[i]);
            }
        }

        // Pass 2: Addition and Subtraction
        let result = pass1[0];
        for (let i = 1; i < pass1.length; i += 2) {
            let op = pass1[i];
            let nextVal = pass1[i + 1];
            if (op === '+') result += nextVal;
            if (op === '-') result -= nextVal;
        }

        return isNaN(result) ? 'Error' : parseFloat(result.toFixed(7)).toString();
    } catch (error) {
        return 'Error';
    }
}

// Core action handler to process calculator tasks from both click and keyboard events
function processAction(actionType, value) {
    if (actionType === 'number') {
        inputNumber(value);
    } else if (actionType === 'operator') {
        if (value === 'calculate') {
            const result = evaluateExpression();
            expression = result;
            evaluated = true;
        } else if (value === 'add') {
            handleOperator('+');
        } else if (value === 'subtract') {
            handleOperator('-');
        } else if (value === 'multiply') {
            handleOperator('×');
        } else if (value === 'divide') {
            handleOperator('÷');
        }
    } else if (actionType === 'action') {
        if (value === 'clear') {
            resetCalculator();
        } else if (value === 'double-zero') {
            inputNumber('0');
            inputNumber('0');
        } else if (value === 'percent') {
            handlePercent();
        }
    }
    updateDisplay();
}

// Event: Standard addEventListener using click event delegation
keys.addEventListener('click', (event) => {
    const { target } = event;
    if (!target.matches('button')) return;

    if (target.classList.contains('number')) {
        processAction('number', target.dataset.value);
    } else if (target.classList.contains('operator')) {
        processAction('operator', target.dataset.action);
```

```
    } else if (target.classList.contains('action')) {
        processAction('action', target.dataset.action);
    }
});

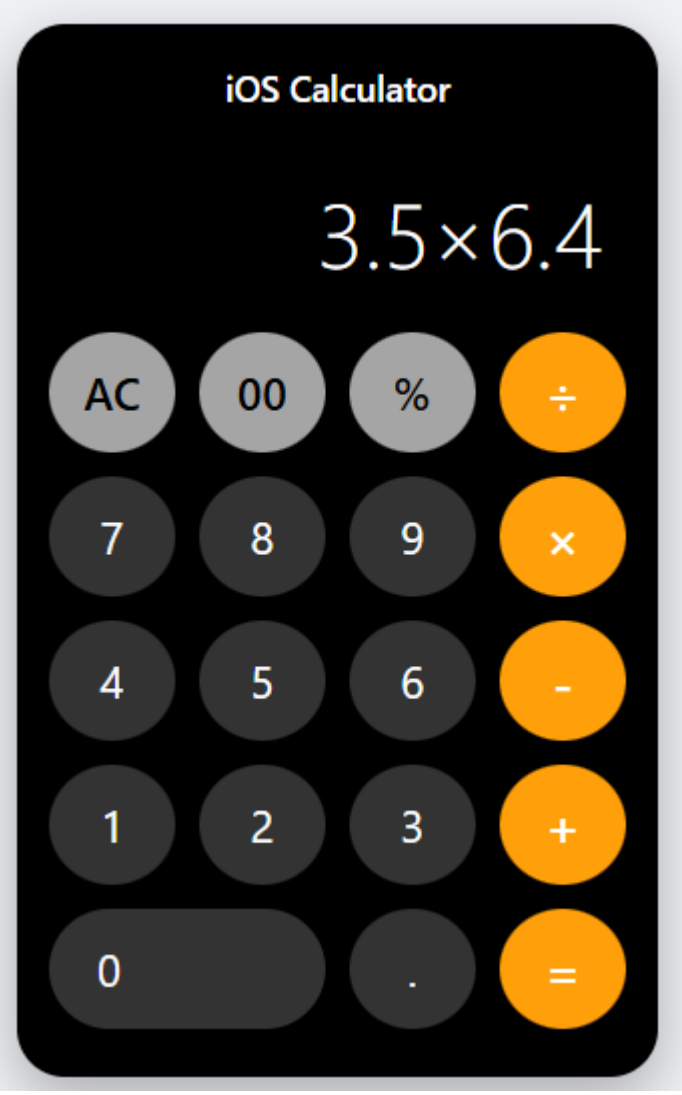
// Event: Standard addEventListener for Keyboard Input
window.addEventListener('keydown', (event) => {
    let key = event.key;
    let targetButton = null;

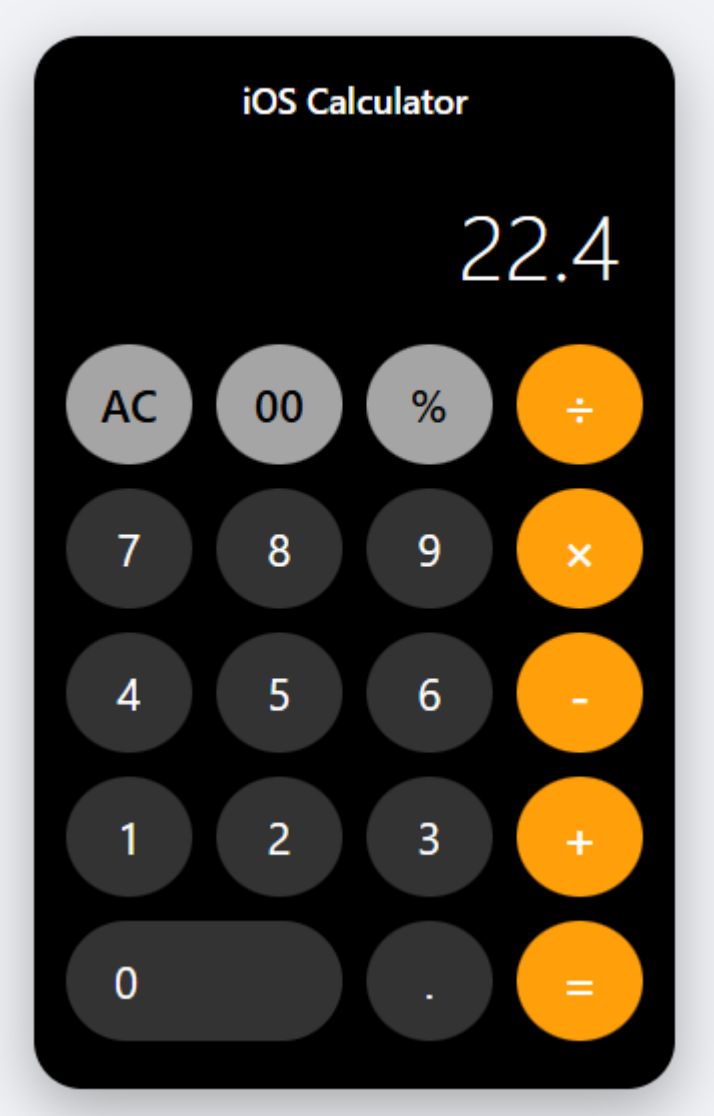
    if (!isNaN(key) || key === '.') {
        targetButton = document.querySelector(`.key.number[data-key="${key}"]`);
        if (targetButton) processAction('number', targetButton.dataset.value);
    } else if (key === '+' || key === '-') {
        targetButton = document.querySelector(`.key.operator[data-key="${key}"]`);
        if (targetButton) processAction('operator', targetButton.dataset.action);
    } else if (key === '*') {
        targetButton = document.querySelector(`.key.operator[data-key="*"]`);
        if (targetButton) processAction('operator', 'multiply');
    } else if (key === '/') {
        event.preventDefault(); // Prevent browser quick-find shortcut
        targetButton = document.querySelector(`.key.operator[data-key="/"]`);
        if (targetButton) processAction('operator', 'divide');
    } else if (key === 'Enter' || key === '=') {
        targetButton = document.querySelector(`.key.operator[data-key="Enter"]`);
        processAction('operator', 'calculate');
    } else if (key === 'Escape') {
        targetButton = document.querySelector(`.key.action[data-key="Escape"]`);
        processAction('action', 'clear');
    } else if (key === 'Backspace') {
        // Delete last character or reset if single digit/error
        if (expression.length > 1 && !evaluated) {
            expression = expression.slice(0, -1);
        } else {
            expression = '0';
        }
        updateDisplay();
    } else if (key === '%') {
        targetButton = document.querySelector(`.key.action[data-key="%"]`);
        processAction('action', 'percent');
    }

    if (targetButton) {
        targetButton.classList.add('keyboard-active');
        setTimeout(() => {
            targetButton.classList.remove('keyboard-active');
        }, 150);
    }
});

// Initial screen render
updateDisplay();
```

Screenshot


iOS Calculator
3.5×6.4
AC
00
%
÷
7
8
9
×
4
5
6
-
1
2
3
+
0
.
=


iOS Calculator
22.4
AC
00
%
÷
7
8
9
×
4
5
6
-
1
2
3
+
0
.
=

Github Link

https://github.com/ManCityP/Web_Development_CSE343